z/OS Job Control Language (JCL)

WHAT IS JOB CONTROL LANGUAGE?

TEKsystems Global Services

WORKFORCE DEVELOPMENT

© 2023 by Innovation In Software Corporation

WHAT IS JOB CONTROL LANGUAGE?

The short answer is that Job Control Language, or JCL, is the language used to enter commands and run programs under the z/OS operating system.

But that answer may not be very informative if you're used to working on operating systems like Unix, Linux, and Microsoft Windows. Those OSes have command line interfaces and scripting languages, but they don't have JCL as such.

In this module, we'll discover what sort of animal JCL is and how it came to be used on IBM mainframes.

© 2023 by Innovation In Software Corporation

A DIFFERENT APPROACH FROM UNIX, WINDOWS

The System 360 came out five years before the start of the ARPANET project. The system predates Unix.

The aspects of operating systems we take for granted today had not been invented yet, and IBM's engineers created their own solutions to the problems of computing.

z/OS today still reflects the design choices made in the 1960s.

© 2023 by Innovation In Software Corporation

SOME CHARACTERISTICS OF IBM MAINFRAME TECH

- Centralized computing
- Batch processing
- Backward compatibility

To get some context about JCL, let's review three fundamental aspects of IBM mainframe technology and how the System 360 of the 1960s evolved into the present-day System Z – centralized vs. networked computing, batch vs. interactive processing, and IBM's commitment to backward compatibility.

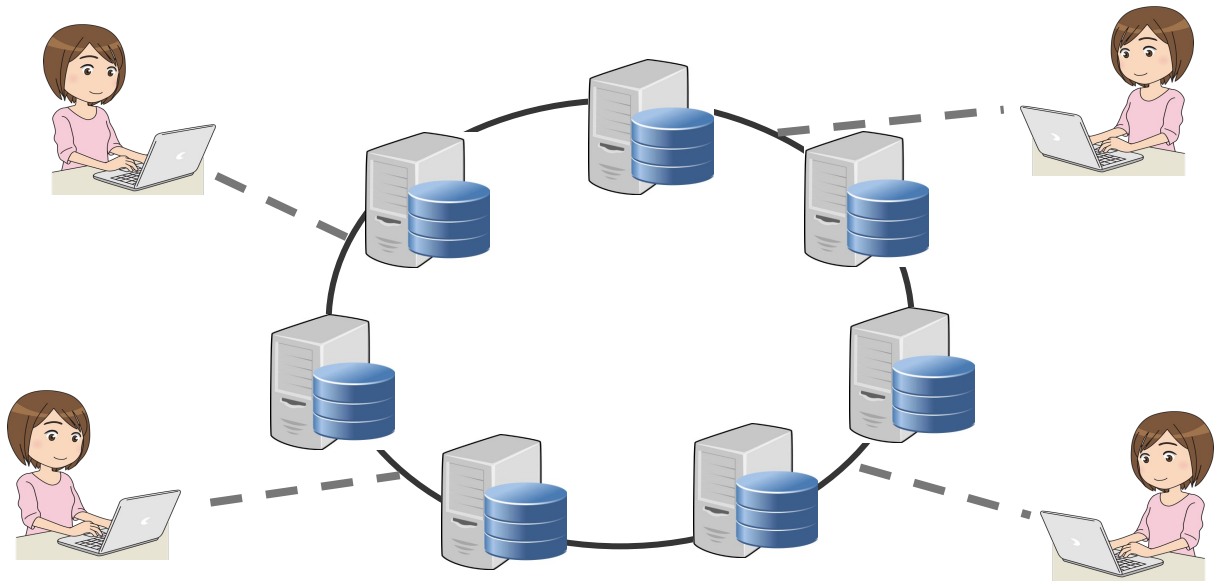
When the System 360 was introduced in 1964, mainframes were the only commercially-available type of computing systems on the market. They were large, heavy, power-hungry, and expensive. They were owned and

© 2023 by Innovation In Software Corporation

CENTRALIZED COMPUTING MODEL

© 2023 by Innovation In Software Corporation

DISTRIBUTED COMPUTING MODEL



The distributed computing model is the norm today, and it has been so since the late 1990s.

We're accustomed to working on an OS instance that may be virtualized or containerized, and that need not manage any other instances on the network.

We connect to remote instances and work on a command line interface or call APIs that give us the illusion we're working locally.

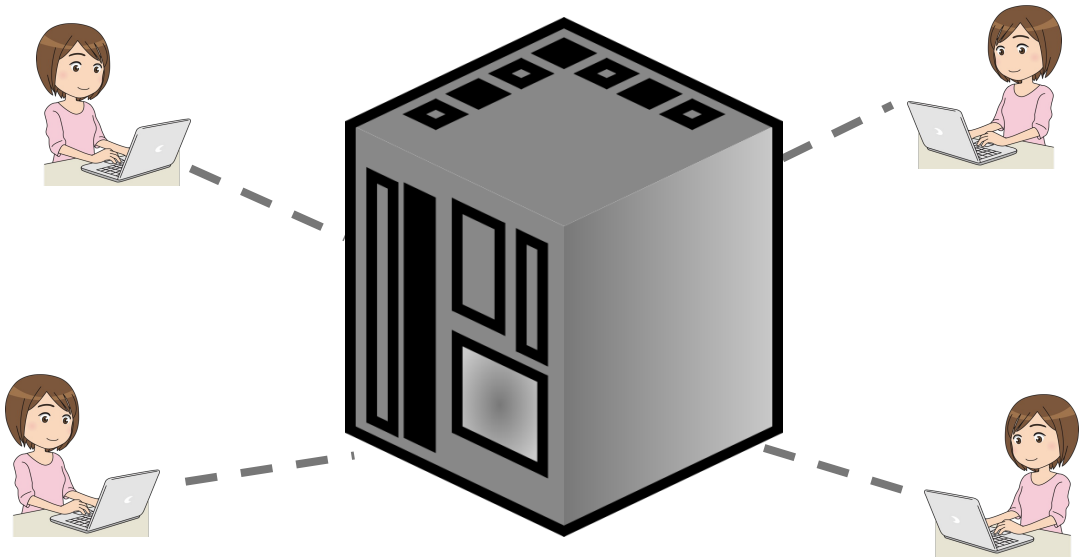
But this is not how things worked when the IBM System 360 was created.

CENTRALIZED COMPUTING MODEL

Networking technology was in its infancy. Operating systems of the day had to perform time-slicing and other algorithms to support multiple tasks and multiple users concurrently.

Users and client systems connected to a host system that managed all the concurrent activity.

The original IBM mainframe operating system, OS/360, was designed around the centralized computing model, and today's z/OS still operates on that principle..

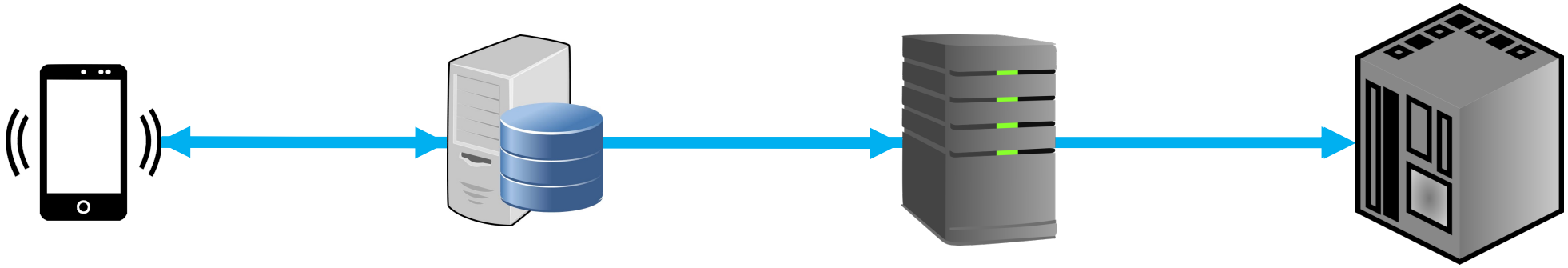


© 2023 by Innovation In Software Corporation

BATCH PROCESSING

© 2023 by Innovation In Software Corporation

TRANSACTIONAL PROCESSING



Today, we normally use computer systems interactively through a series of transactions.

Think about how you manage your bank accounts through your smart phone, for instance.

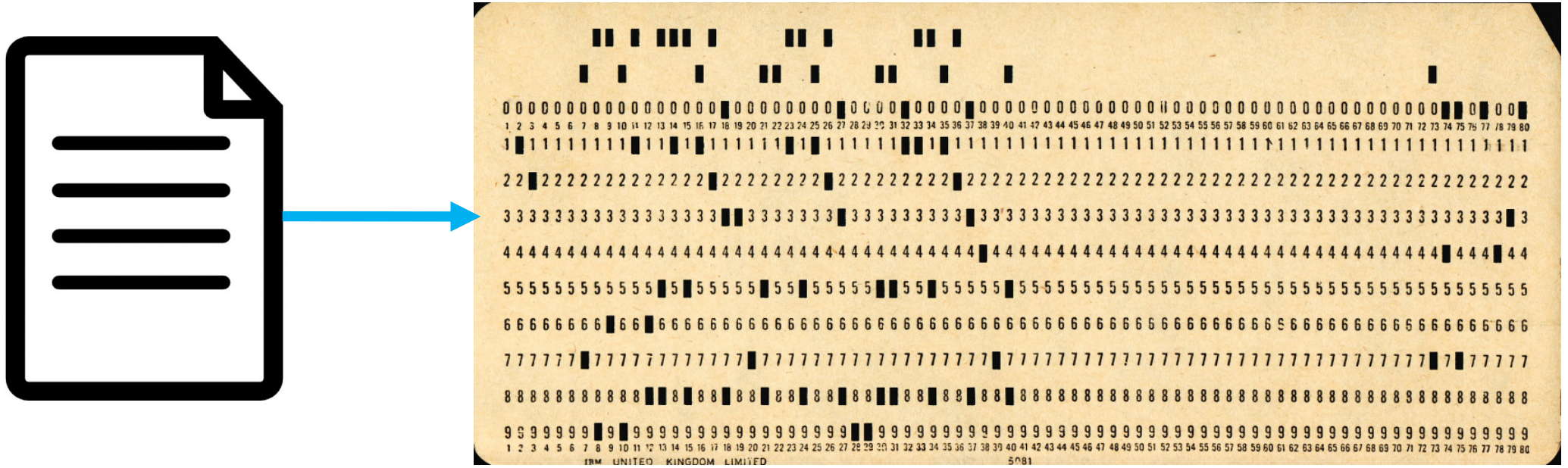
You might effect a funds transfer operation by initiating a transaction that touches several computer systems, from your phone to a Web server to an application server to a back-end system that may well be a mainframe, with queries reaching out to other systems to perform authentication and authorization, check for fraud and verify account statuses and balances, and so forth.

© 2023 by Innovation In Software Corporation

BATCH PROCESSING

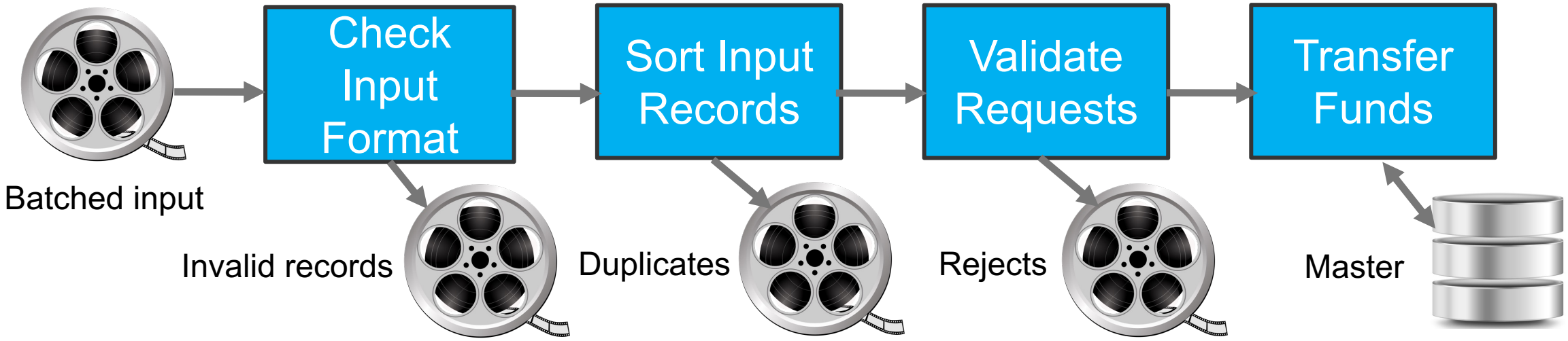
Back in the 1960s, you would have requested the funds transfer using a paper document.

The request would be encoded by clerical staff at the bank into a form their mainframe could consume, such as punched cards, paper tape, or magnetic tape.



© 2023 by Innovation In Software Corporation

BATCH PROCESSING



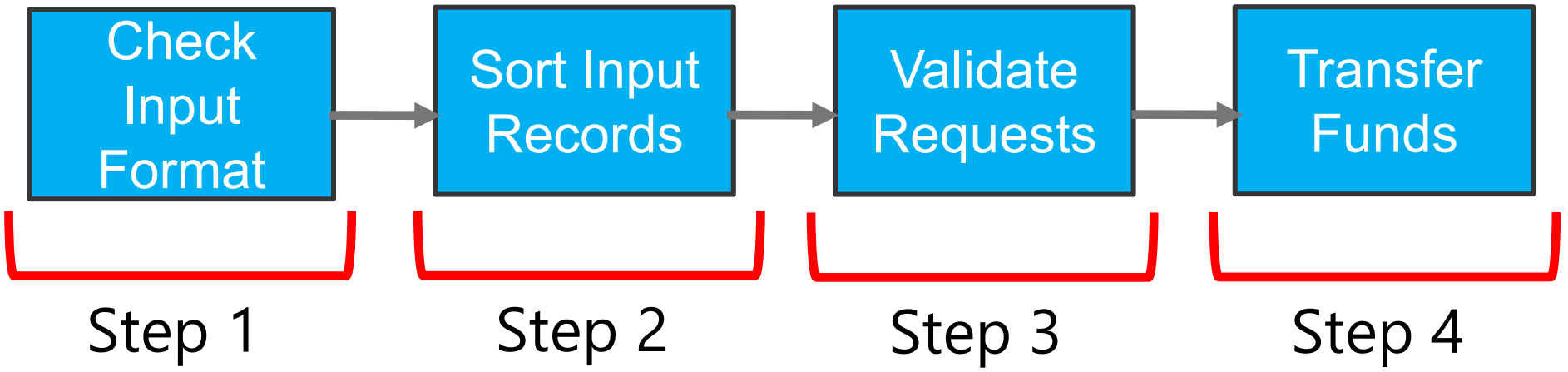
Overnight, technical staff would run a sequence of programs to read all the funds transfer requests from all customers and process a large set of them together – a *batch*.

The four steps in this example would not happen for each input record one by one. Instead, step 1 would process all the records, then step 2 would process all the records that survived step 1, step 3 would process all the records that survived step 2, and so on..

This mode of processing is called "batch processing." It is the native operating mode of IBM mainframes, and JCL is the native mainframe language for controlling it.

© 2023 by Innovation In Software Corporation

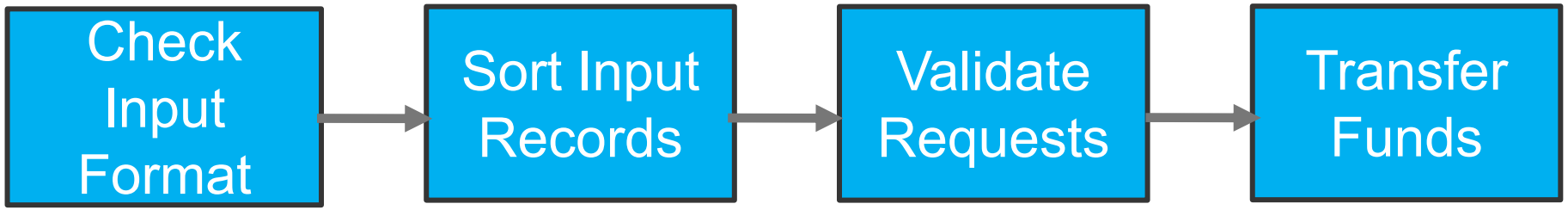
BATCH PROCESSING



The programs that run in sequence are called "job steps" or just "steps."

© 2023 by Innovation In Software Corporation

JCL AND BATCH PROCESSING




```
//FNDXFER  JOB
//* VERIFY INPUT RECORD FORMAT
//CHECK    EXEC PGM=...
//INFILE   DD DSN=...
//PASSED   DD DSN=...
//INVRECS  DD DSN=...
//* SORT/MERGE INPUT RECORDS
//SORT     EXEC PGM=ICEMAN
//SYSUT1   DD DSN=...
//SYSUT2   DD DSN=...
```

etc.

JCL controls the initiation and execution of jobs.

There are four basic statements:

- The JOB statement defines a job.
- The comment statement describes intent.
- The EXEC statement defines a step.
- The DD statement defines a data set or file.

© 2023 by Innovation In Software Corporation

BACKWARD COMPATIBILITY

© 2023 by Innovation In Software Corporation

BACKWARD COMPATIBILITY

IBM 702 - 1953

We're all familiar with the idea of breaking and non-breaking changes, and how the Semantic Versioning scheme is meant to ensure breaking changes only occur with major version number increments.

Backward compatibility in the mainframe world goes far beyond that.

When the IBM System 360 was introduced, each computer system was unique.

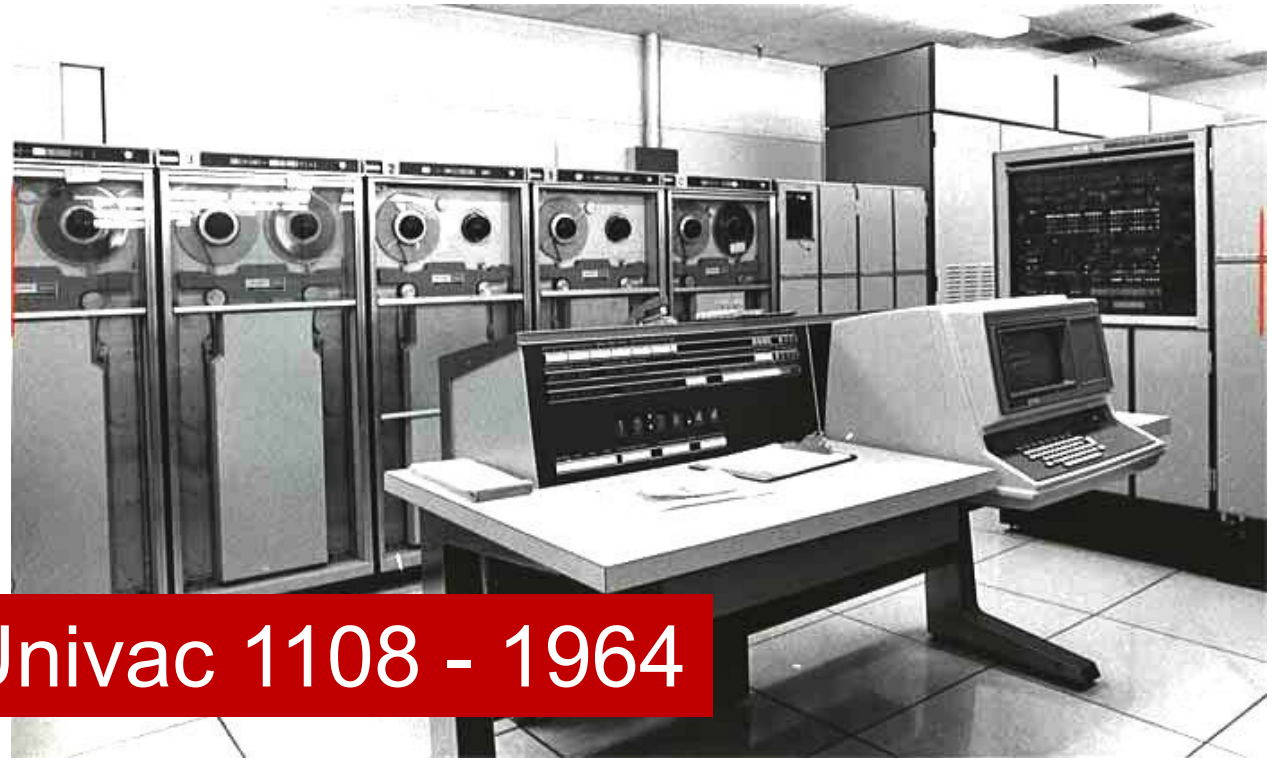
© 2023 by Innovation In Software Corporation

BACKWARD COMPATIBILITY

Each system – even those from the same vendor – had its own hardware architecture, instruction set, operating system, programming languages, and file systems.

© 2023 by Innovation In Software Corporation

BACKWARD COMPATIBILITY



Univac 1108 - 1964

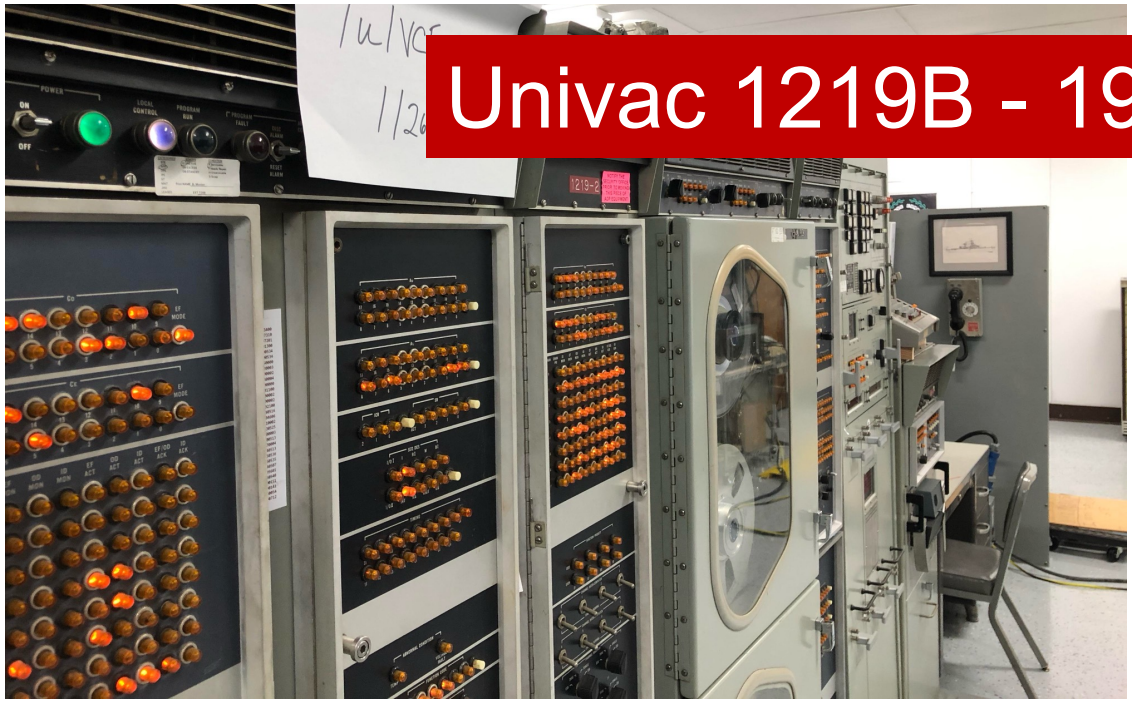
When a company needed to upgrade their computer system, they had to discard everything they had in place and re-write all their applications for the new system.

© 2023 by Innovation In Software Corporation

BACKWARD COMPATIBILITY

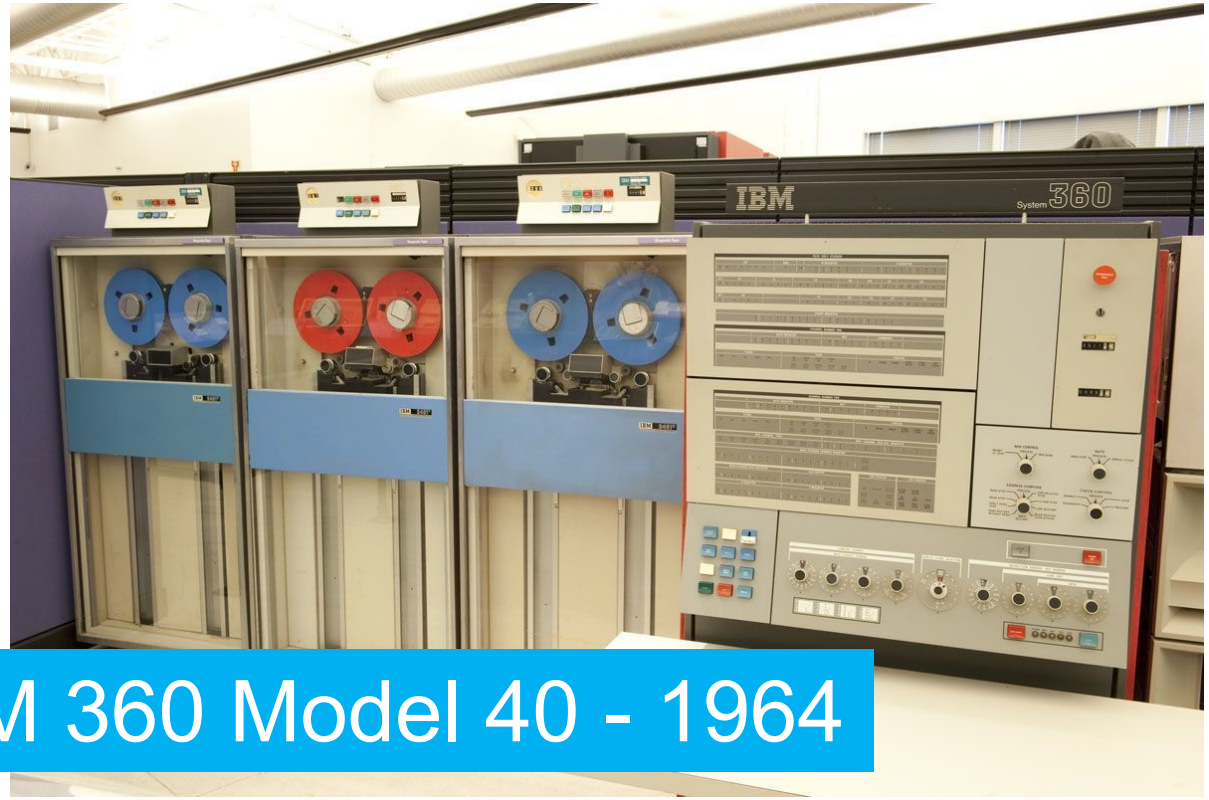
Computers had quickly gone from a novelty used in only a few large corporations to a baseline requirement for business success. Improvements in the technology were moving at breakneck speed.

The cost of keeping up was becoming untenable, as each move forward involved a complete replacement of the hardware and software a company had in place.



© 2023 by Innovation In Software Corporation

BACKWARD COMPATIBILITY



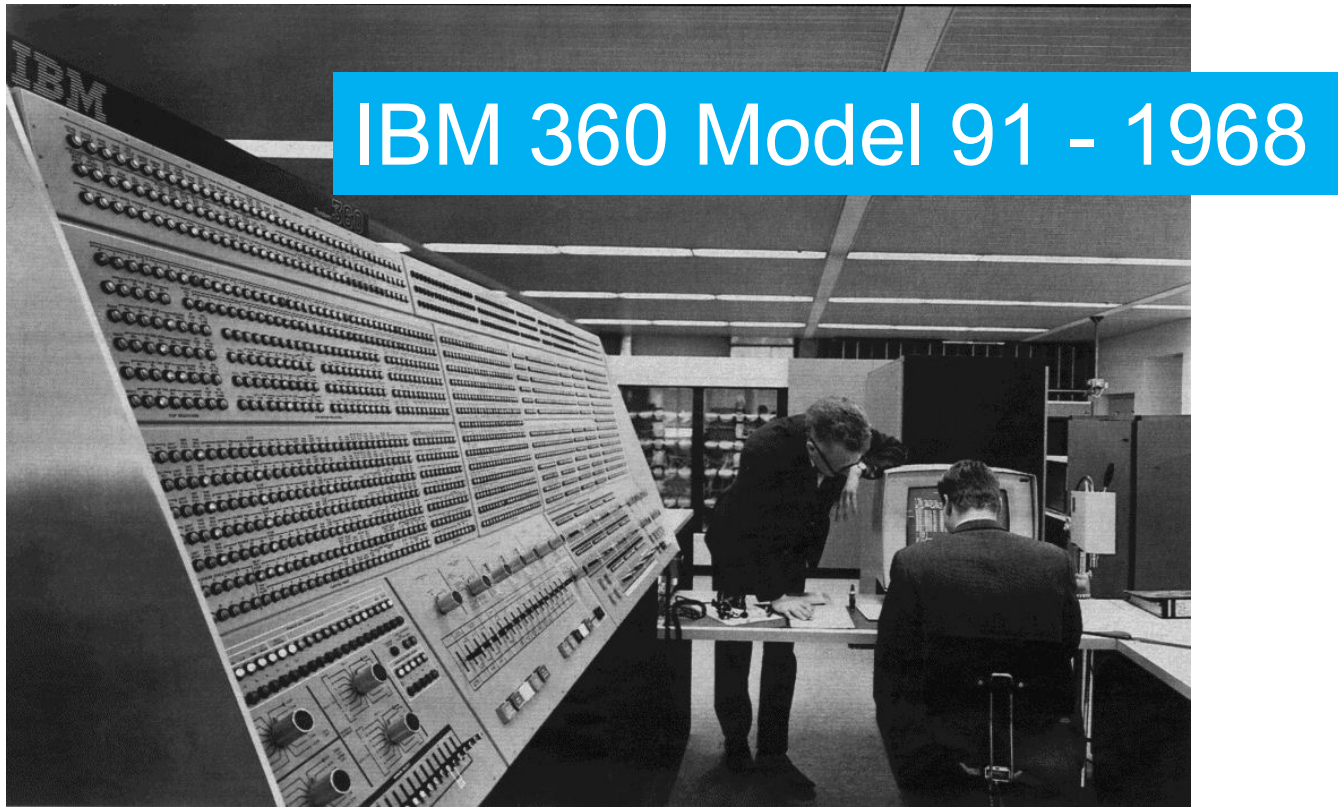
IBM 360 Model 40 - 1964

With the introduction of the System/360 in 1964, IBM announced a commitment to backward compatibility they still honor to the present day.

© 2023 by Innovation In Software Corporation

BACKWARD COMPATIBILITY

IBM wanted customers to be able to upgrade their systems without losing their investment in existing applications.



© 2023 by Innovation In Software Corporation

BACKWARD COMPATIBILITY

IBM eServer zSeries - 2000

Backward compatibility isn't only about controlling when breaking changes may be introduced.

© 2023 by Innovation In Software Corporation

BACKWARD COMPATIBILITY

Backward compatibility means an executable that was compiled and linked on a System 360 in the year 1964 will still run on a System Z built in the 2020s.

© 2023 by Innovation In Software Corporation

BACKWARD COMPATIBILITY

IBM z16 Rack Mount - 2022

At the same time, IBM has kept up with – and in many cases taken the lead in – every new development in computing for the last 50 years.

The System Z supports all of it – from the original System 360 design elements to the latest and greatest capabilities of contemporary computer systems.

© 2023 by Innovation In Software Corporation

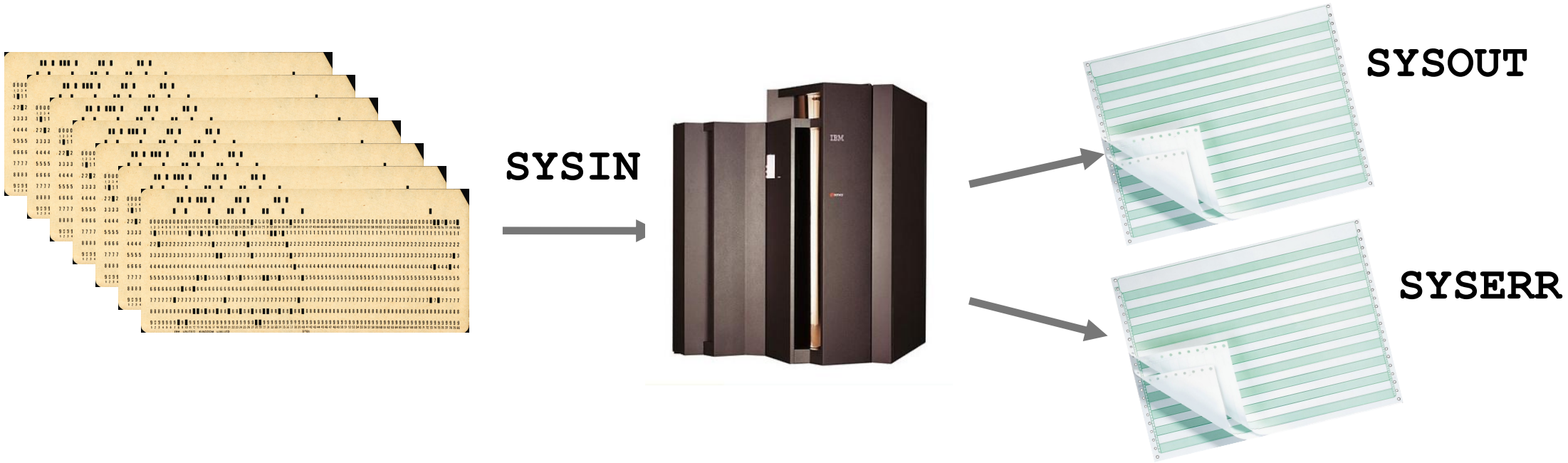
LAYERING NEW ON TOP OF OLD

2010s	Docker, Kubernetes
2000s	64-bit addressing, REST, Linux, Java
1990s	ASCII, POSIX, Unicode
1990s	TCP/IP, HTTP, SOAP, Sysplex
1980s	31-bit addressing, DBCS
1960s	24-bit addressing, EBCDIC, SNA

The way IBM has managed to modernize the mainframe year by year without compromising backward compatibility is exactly as you probably imagine – they have layered new on top of old.

© 2023 by Innovation In Software Corporation

REMNANTS OF PUNCHED CARDS & LINE PRINTERS



The oldest of the old is the batch-oriented design of the original mainframe. JCL was the way work was loaded into a mainframe in 1964, and it is the way work is loaded into an IBM Z today. JCL is coded in 80-byte records that look like the 80-column punched cards used with the original System 360.

z/OS reads the system input stream as if it were reading 80 column punched cards. It writes the system output streams as if it were writing to a line printer with 132 character lines.

SUPPORT FOR INTERACTIVE PROCESSING

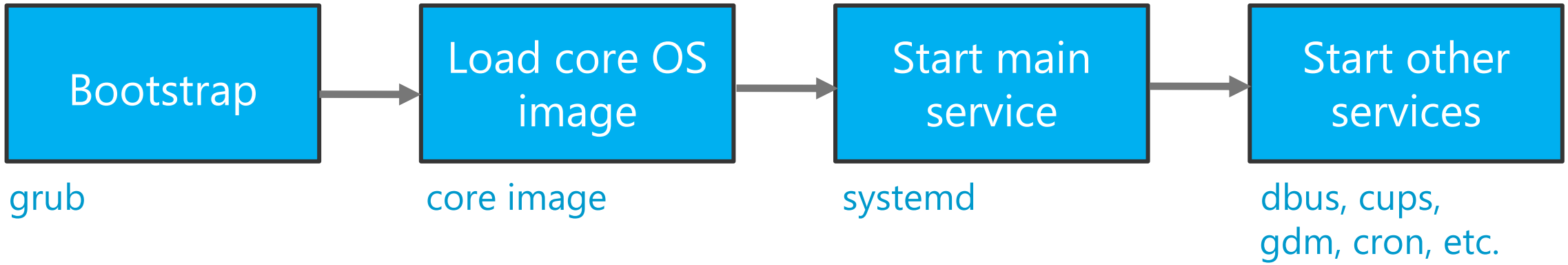
- VTAM – Virtual Telecommunication Access Method
- TSO – Time Sharing Option
- USS – Unix System Services
- CICS – Customer Information Control System
- IMS/TM – Information Management System/Transaction Manager

Support for long-running processes and interactive processing was built on top of support for batch processing. Services such as network support – VTAM – and interactive facilities like TSO and CICS, which you'll be learning about in this program, are initiated with JCL.

As far as z/OS is concerned, they are batch jobs. Instead of reading a large file containing similar records, like the funds transfer job, this type of job stays active and waits for interactive requests to arrive. But under the covers it is still fundamentally a batch job.

© 2023 by Innovation In Software Corporation

GENERAL OPERATING SYSTEM START-UP (E.G., LINUX)



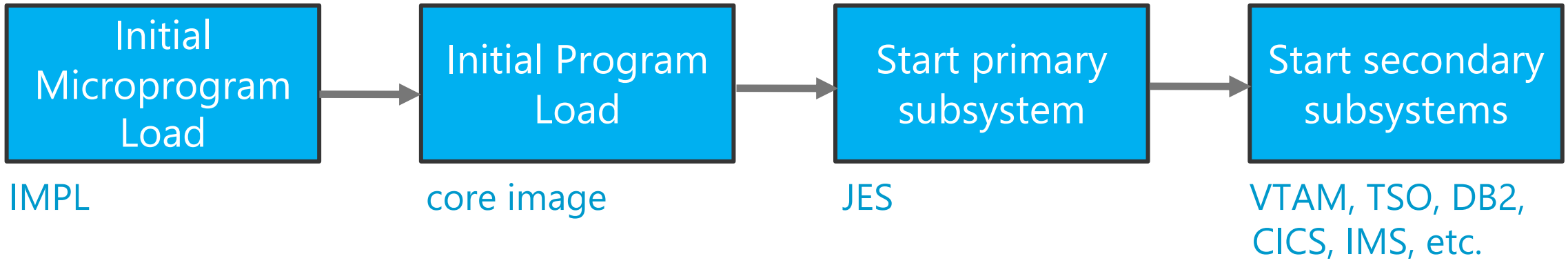
For context: All operating systems start up by loading an executable image of core functionality, read from a predefined location on external media by a small piece of code that only performs that one task.

The image initializes the heart of the OS – the Microsoft Windows core or the Linux kernel, for instance.

Then an initialization process loads whatever services the OS requires to support normal operation. This may be a series of scripts – like initd – or a main service that then handles loading other services – like systemd.

© 2023 by Innovation In Software Corporation

Z/OS START-UP



z/OS starts up in a similar way, but the entire system is batch-oriented. The primary subsystem is always the Job Entry Subsystem, or JES. That is the software that interprets and acts upon Job Control Language.

Everything that happens on a z/OS system – including long-running processes that perform services in the background or that support interactive processing or OLTP – is initiated through JCL.

Understanding and writing JCL is the most fundamental skill you will need to work effectively on this platform. Without it, you can't do much with a mainframe besides look at it.

© 2023 by Innovation In Software Corporation

WHAT YOU KNOW

- At its heart, the System Z is a batch processor.
- Support for interactive use and transaction processing are added on top of batch support.
- Job Control Language is the language for initiating and managing batch jobs.
- JCL statements still follow the same format as punched cards from the 1960s.
- The Job Entry Subsystem (JES) is the Primary Subsystem on z/OS, and is responsible for interpreting JCL and managing jobs.

© 2023 by Innovation In Software Corporation

WHAT'S NEXT

- JCL statement format